Algorithm's design techniques

Recursion and backtracking

Recursion is a classical way of solving some type of problems in programming, specially Divide and conquer -type of problems. Recursion suits specially to problems that the solution requires to execute the same function several times but with different inputs. The idea is that the function calls itself until a condition (called base case) is met. This is usually the first thing to define, to avoid a common problem like infinite recursion.

A very typical example problem solved with recursion is the calculation of the factorial of a number. The factorial of a number n is defined as $n \cdot (n-1) \cdot (n-2) \cdot ... \cdot 3 \cdot 2 \cdot 1$. This can be rewrite as $factorial(n) = n \cdot factorial(n-1)$. This is recursion, the solution includes the problem itself.

To implement this function, the first thing should be to define the base case:

```
def factorial(n):
    # Base case
    if n == 1:
        return 1
```

After the base case, that limits the recursion, is defined, we can add the main expression to the function:

```
def factorial(n):
    # Base case
    if n == 1:
        return 1
    # Common case
    return n * factorial(n-1)
```

Recursion is just a technique that can help in some situations but is good to remember that it is always possible to implement an iterative version of a recursive function. Depending on the problem it can be easier or harder, but there is always an iterative version. And in the same way, in some situations is possible find a recursive version for an iterative function. Some problems are more suitable to recursion than others and sometimes the recursion version is what comes naturally to mind.

A possible problem with recursion is that each recursive call stores the context of the function on the stack in memory, so it consumes memory space. Programming languages limit recursion to avoid stack overflow errors. Technically infinite recursion is not possible due to these limits, but if the base case is not adequately set, it is going to lead to an error or exception.

Backtracking is a form of recursion, where a certain "path" is followed when doing the recursion, and we can go back one or more recursion levels to follow a different path or to go back to the origin. A typical example is graph traversing.

Another practical example is to find the permutations of a given data:

```
def permutations(data, n=0):
    if n == len(data)-1:
        return data
    return [pattern + c for pattern in permutations(data, n+1) for c in data]
```

You can call this function with:

```
permutations('123')
# or
permutations(('1', '2', '3'))
```

There is nothing wrong with recursion, but it is good thing to try to solve problems also in a more direct and iterative way. This is a little bit a silly example, but if we try to implement a function to resolve this:
$f(n) = \sum_{k=0}^{n-1} n - k = n + (n-1) + (n-2) + ... + 3 + 2 + 1$, we will be tempted to use recursion, or a loop in a iterative solution, but this particular problem can easily be solved with a mathematical expression: $f(n) = \frac{n(n+1)}{2}$

This was just a reminder that always is better to think, a little bit at least, about different solutions for the problem.

Algorithm strategies

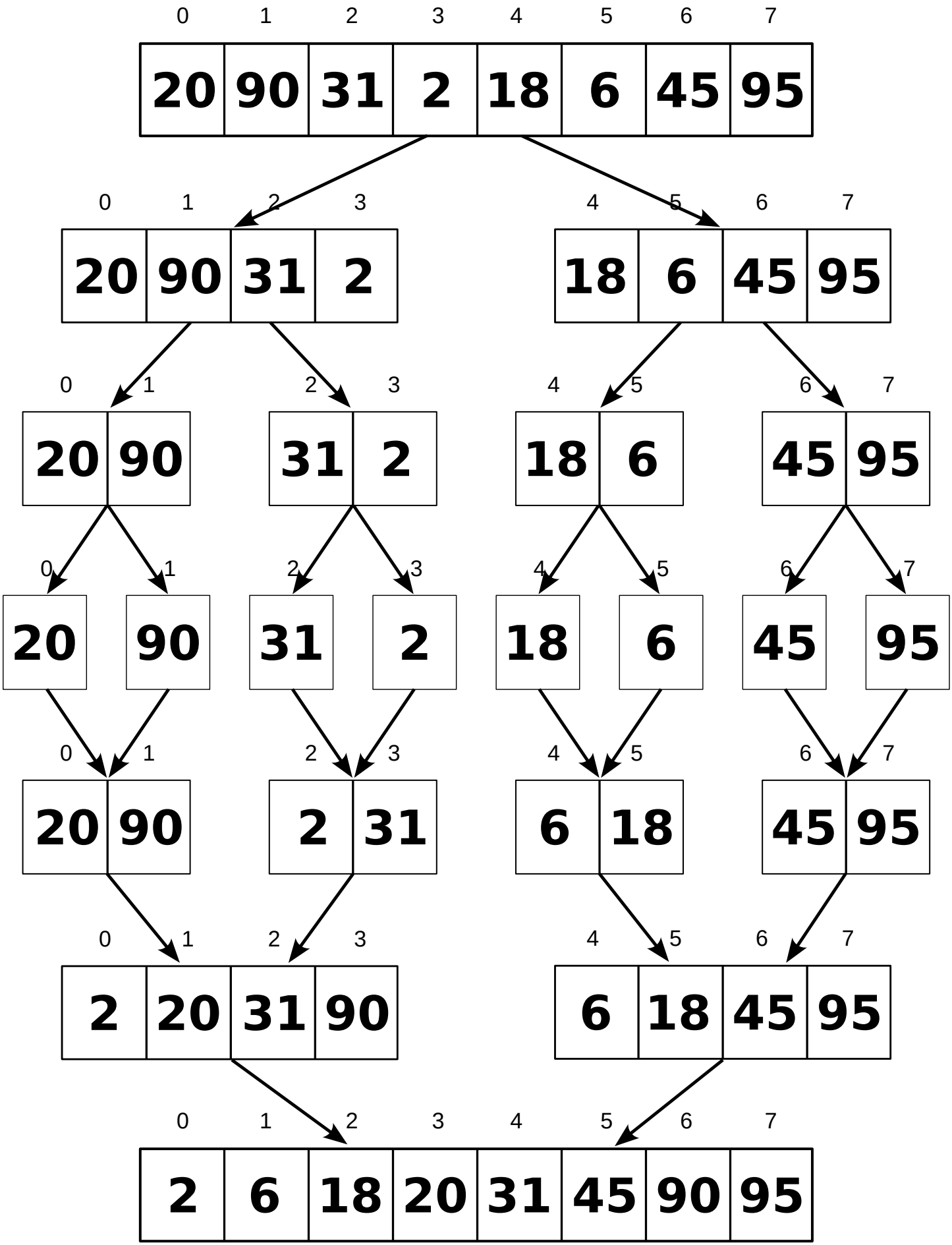
There are different strategies on how to try to resolve some given problem. We are going to see a little bit of some of them

Divide and conquer

Divide and conquer is one of the typical strategies that can be used in certain situations. The idea is to keep dividing the problem into smaller problems until they are directly solvable (the conquer part) and then start recombining/merging the solutions until we get the solution of the original problem.

Merge sort

Merge sort is a classic example algorithm of the "Divide-and-conquer" family. In this case the idea is to split the array or partition in half and call recursively on each half. The base case for the function is when the size of the array is just one element. If that is the case return that array, that is considered sorted. With that, the previous level should merge (hence the name) the two halves into one. That can be achieved with two pointers, one on each half. The values on the pointers are compared and the smaller one is used to build the new array. The pointer is increased when the value is used. The operation is repeated until one of the halves is exhausted, then the rest of the other half is added to the resulting array and this array is then returned to the previous level of the calling stack.


0 1 2 3 4 5 6 7
20 90 31 2 18 6 45 95
0 1 2 3 4 5 6 7
20 90 31 2
18 6 45 95
0 1 2 3 4 5 6 7
20 90
31 2
18 6
45 95
0 1 2 3 4 5 6 7
20
90
31
2
18
6
45
95
0 1 2 3 4 5 6 7
20 90
2 31
6 18
45 95
0 1 2 3 4 5 6 7
2 20 31 90
6 18 45 95
0 1 2 3 4 5 6 7
2 6 18 20 31 45 90 95

The instructions already given should be enough for you to implement your own version of Merge sort, as they are almost step by step instructions on what the function does.

Exercise!

Given the information you have received about the Merge sort algorithm, try to implement it. Given an array like: `[6, 8, 5, 1, 2]`, the sorted array would be: `[1, 2, 5, 6, 8]`

Don't continue until you have finished the exercise!

We can test the algorithm in the same way we did in the sorting chapter:

```
array=[randint(0,10000) for _ in range(10000)]
start_time = time.time()
_ = merge_sort(array)
runtime = time.time() - start_time
print(f'Merge sort runtime: {runtime}')
```

And that gives a runtime of 0.036 seconds on the computer this course has been written on. Compared to other sorting algorithms, it performs quite well and its code is not particularly complex. It's not a surprise that merge sort has a time complexity of $\theta(n \cdot \log n)$ for the average case, given that it is based in splitting in half the partitions. As we have seen before, that means that the depth level of recursivity is $\log n$. And given that in each level around n operations are performed, we can understand the time complexity value.
This algorithm also has a best case and worst case of $O(n \cdot \log n)$, making it also an efficient and predictable algorithm.

Merge sort is not an on-place algorithm, as it needs as much extra space as the array itself. A version of this algorithm using a linked list instead of an array needs only one extra node, but then it can't benefit from the $O(1)$ indexing time complexity of an array.

More on this algorithm: Wikipedia entry for Merge sort

Dynamic programming

Same as Divide and conquer, **Dynamic programming** is based on the assumption that a problem can be broken into subsmaller problems that can be solved and recombined to solve the original problem. But Dynamic programming suits better for when the subproblems can overlap with each other. So getting the solution to one of them can also help to solve other "subproblems". That means that these algorithms use some kind of caching or remembering technique to not to recalculate an already solved subproblem. This type of programming usually uses recursion, but it is not a requirement, also iterative implementations are possible.

The techniques to remember the solutions can be different. One of them is **Memoization**. In this technique, once a subproblem is solved, its input and solution are stored somehow and the next time a subproblem with the same input is presented, it just has to retrieve the already computed result from where it is stored. The possible problem with this technique is that as the input has to be stored along the solution, it is not suitable for problems with large inputs.

On the other hand, in **Tabulation** the algorithm is designed and built from the beginning around the idea of benefiting from the already precomputed solutions, so usually this type of algorithms are not recursive but iterative. In comparison to Memoization it doesn't need to store large quantities of data. The algorithm stores just the minimum and necessary to compute the next step.

One known problem and classical example for students that can benefit from dynamic programming is the Fibonacci problem. The Fibonacci series are a Mathematical series that can be defined as: $fib(n) = fib(n-1) + fib(n-2)$

With two base cases defined as $fib(1) = 1$ and $fib(0) = 1$. So, for example, $fib(5)$ will be computed as $fib(4) + fib(3)$. This problem is by definition a recursive problem (but it can also be solved iteratively). And a fast implementation could be like this:

```
def fib(n):
    if n < 2:
        return 1
    else:
        return fib(n-1) + fib(n-2)
```

This can work quite well for small integers. For example `fib(10)` is executed almost instantly, but `fib(40)` takes up to 30 seconds in my computer and `fib(50)` can take some hours (not tested, but it would take like 5 hours given the progression). The problem is of course that it keeps recalculating the same operations again and again. And with a quick analysis of the algorithm, we can understand

that the time complexity of it is around $O(2^n)$ (from one call, comes 2 calls. From them come 4 calls in total. From them come 8 calls in total...)

So here is where these techniques come in handy. Memoization tries to remember the solutions. So we could add a cache structure to the function to store the computed solutions and use it first to try to see if there is an already computed solution available. The cache structure should have a time complexity of $O(1)$ for access, as we are going to query it a lot. An array or a hash dictionary can do the trick.

```
def fib(n, cache={0:1, 1:1}):
    if len(cache) <= n:
        cache[n]= fib(n-1, cache) + fib(n-2, cache)
    return cache[n]
```

Notice

Be very careful when defining this way a parameter like `cache` in the example. In the example the `cache` parameter gets its value at **function definition**, even before the function has been called for the first time. And the value is remembered across function calls. In this case this goes to our advantage, because if we calculate `fib(10)`, the cache array is filled with ten values. And when we call `fib(20)`, those values are already there and do not need to be recalculated.

But take in mind that some other cases this is not the desired behavior, and you want to get the default value of `cache` each time you call the function. In that case, you have to define its value inside the function.

Python even includes a module for this cache functionality. In old versions there was `functools.lru_cache`, and from Python 3.9+ there is `functools.cache`. It is used as a decorator for the function we want to cache.

```
from functools import cache

@cache
def fib(n):
    if n < 2:
        return 1
    else:
        return fib(n-1) + fib(n-2)
```

For this reason, some authors consider that Memoization is not a Dynamic programming technique but rather a cache technique.

Notice that even if the result for relatively big values of n can be calculate quickly, you can easily hit the limit for recursive calls in Python, that is set at 1000 recursive calls. You can calculate bigger values of the Fibonacci series (for example 5000), but you have to call the `fib()` function several times with bigger values each time to fill up the cache and not hit the recursion limit.

The Tabulation version is built in a different way. Tabulation versions are built bottom-up. That means, that instead of trying to calculate first the requested value and recursively call up-bottom until the base case, we calculate the value bottom-up, that is, iterating in a loop while calculating the values at each step with help of the necessary precalculated values. In the Fibonacci case, that means, that we iterate from 0 up to `n` calculating Fibonacci at each step, but using the stored values (in variables, for example) of the two previous steps.

Go to the Moodle environment and try to implement this version on your own.

Exercise!

Given the information you have received about Dynamic programming and Tabulation, try to implement a function to solve the Fibonacci series with this technique.

Don't continue until you have finished the exercise!

A possible solution is:

```
def fib(n):
    if n < 2:
      return 1
    prev = prev_prev = 1
    for _ in range(n-1):
        result = prev + prev_prev
        prev, prev_prev = result, prev
    return result
```

With this version, you can easily calculate the result of the Fibonacci series for big values of n. For example, in my computer, `fib(1000000)` is calculated in about 9 seconds. Another benefit of Tabulation is that it uses less memory than Memoization. It just stores the two previous values in memory.

Greedy algorithms

For certain type of complex problems, usually with multiple steps, a popular approach are **Greedy algorithms**. Greedy algorithms are based on the idea of applying the best optimal solution for every local step in the process, with the hope that it will give us the best global solution. This is not always true, so these kinds of algorithms must be examined to check if the resulted global solution is the most optimal solution for the problem. On the other hand, these algorithms are usually easy and fast to implement and they will give us some result, at least.

Probably it's better to see an example. Given a series of coin denominations of some currency, we want to get the minimum number of coins to account for certain amount. If in this currency, there are coins of face value of 8, 5 and 1. Greedy algorithm will suggest to take the coins that take us to the wanted amount faster without going over. For example if we want to get an amount of 15, first we would take a coin with face value of 8 (two would go over 15). Then we would take a coin with face value of 5 (with two of them it would go over 15). And finally, we would take 2 coins with face value of 1.

So, if we want to implement an algorithm for this, we can calculate for each coin face value (from biggest to smallest) how many of them can be used without going over. This is just a matter of dividing (without decimals) the wanted amount between the face value of the coin. Subtract the already calculated quantity from the original amount and repeat for the next smaller value.

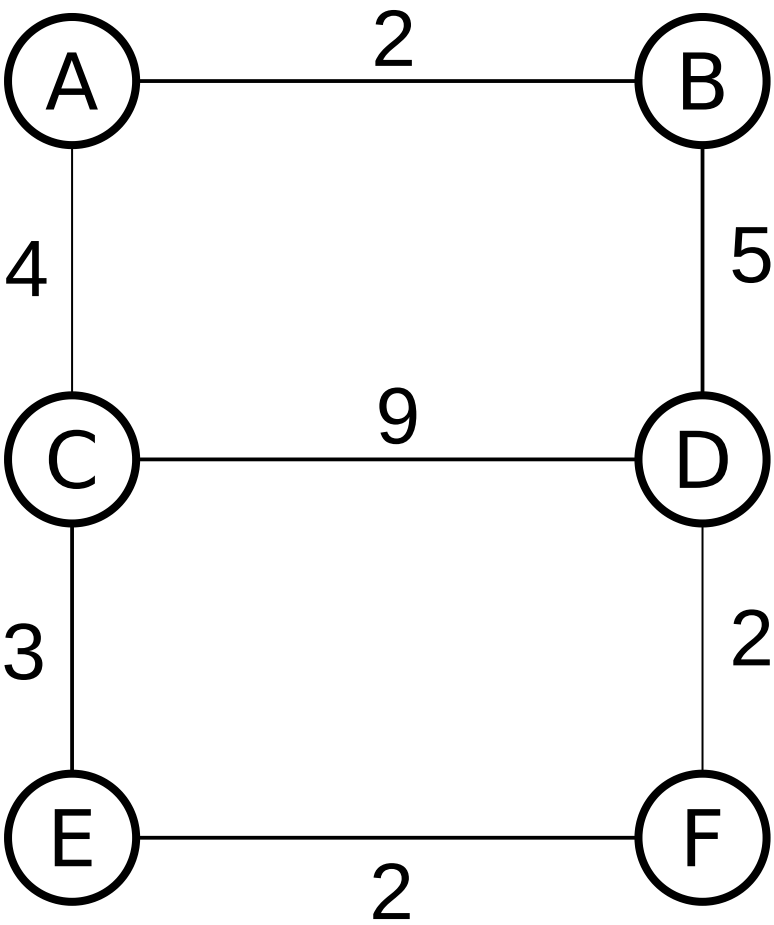
A possible implementation could be like this:

```python
def min_coins(face_values, amount):
    result = []
    for face_value in sorted(face_values, reverse=True):
        calculated = amount // face_value
        amount -= calculated * face_value
        result.append(calculated)
    return result
```

So, as we see, this algorithm gives us a quick result quite easily. The problem with Greedy algorithms is that very much care has to be taken to be sure that the results are accurate. In our example, we have different values, where our Greedy algorithm fails. For example, for the same face values and an amount of 12, we get the result of [1, 0, 4]. In total 5 coins. But actually, we can get a better result [0, 2, 2], with a total of 4 coins. Another example is 52 ([6, 0, 4] vs [4, 4, 0]).

Anyway, Greedy algorithms usually gives us a fast result and quite close to the best solution. That is the reason they are used with some problems. One of the most known examples is the **Dijkstra**'s shortest path algorithm for graphs.

Let's see how it works with the following graph as an example:



Given the structures we saw in the graph's chapter, we can create the Python object for this graph with:

```python
from Graph import Vertex, Edge, Graph

# Create the vertices
A = Vertex('A')
B = Vertex('B')
C = Vertex('C')
D = Vertex('D')
E = Vertex('E')
F = Vertex('F')

# Create the edges
AB = Edge(A, B, 2)
AC = Edge(A, C, 4)
BD = Edge(B, D, 5)
CD = Edge(C, D, 9)
CE = Edge(C, E, 3)
DF = Edge(D, F, 2)
EF = Edge(E, F, 2)

# Create the adjacent map for this graph
adj_map = {
    A: { B: AB, C: AC },
    B: { A: AB, D: BD },
    C: { A: AC, D: CD, E: CE },
    D: {B: BD, C: CD, F: DF},
    E: {C: CE, F: EF},
    F: {D: DF, E: EF}
}

# Create the graph based on the adjacent map
g = Graph(adj_map)
```

The Dijkstra's shortest path algorithm will initialize a structure where to store the discovered shorted path to different vertices from the source vertex. Something like:

```
{
    <Vertex: A>: {'shortest': inf, 'previous': None},
    <Vertex: B>: {'shortest': inf, 'previous': None},
    <Vertex: C>: {'shortest': inf, 'previous': None},
    <Vertex: D>: {'shortest': 0,   'previous': None},
    <Vertex: E>: {'shortest': inf, 'previous': None},
    <Vertex: F>: {'shortest': inf, 'previous': None}
}
```

The source vertex is obviously marked with a shortest distance of 0. The rest have an *infinite* value as shortest discovered distance (the *infinite* value can be obtained in Python with `float('inf')`).

This structure also stores from where comes the shortest path. At first this is initialized to None for all vertices.

Next, we have a list of the vertices that are still not visited. At first, of course, this is all vertices. The algorithm will loop while there are still vertices in the list. And now comes the greedy part of the algorithm. It will select the vertex with the minimum shortest distance in the structure we saw before and try to go forward from there calculating the shortest paths to the different adjacent vertices that are still unvisited. For example, in the first iteration, the selected vertex will be D, as has the shortest path, and the algorithm will calculate and update the path to its adjacent vertices, if they are shorter than the current distance. So, after the first round we have something like:

```
{
    <Vertex: A>: {'shortest': inf, 'previous': None},
    <Vertex: B>: {'shortest': 5,   'previous': D   },
    <Vertex: C>: {'shortest': 9,   'previous': D   },
    <Vertex: D>: {'shortest': 0,   'previous': None},
    <Vertex: E>: {'shortest': inf, 'previous': None},
    <Vertex: F>: {'shortest': 2,   'previous': D   }
}
```

At the end of the loop the current vertex D is removed from the list of unvisited vertices. Then, it will loop again doing exactly the same. In this case, the selected vertex will be vertex F, as it has the shortest distance. And now the adjacent vertices of this vertex will be checked to try to update their shortest distance with one through F. D will not be checked as it is already visited. After this second round, the structure will be like:

```
{
    <Vertex: A>: {'shortest': inf, 'previous': None},
    <Vertex: B>: {'shortest': 5,   'previous': D   },
    <Vertex: C>: {'shortest': 9,   'previous': D   },
    <Vertex: D>: {'shortest': 0,   'previous': None},
    <Vertex: E>: {'shortest': 4,   'previous': F   },
    <Vertex: F>: {'shortest': 2,   'previous': D   }
}
```

F is removed from unvisited vertices list and the next round vertex E is selected as the unvisited vertex with shortest distance. After updating its unvisited adjacents, an interesting thing happens. The vertex C gets updated from a distance of 9 and a path from D, to a distance of 7 and a path from E:

```
{
    <Vertex: A>: {'shortest': inf, 'previous': None},
    <Vertex: B>: {'shortest': 5,   'previous': D   },
    <Vertex: C>: {'shortest': 7,   'previous': E   },
    <Vertex: D>: {'shortest': 0,   'previous': None},
    <Vertex: E>: {'shortest': 4,   'previous': F   },
    <Vertex: F>: {'shortest': 2,   'previous': D   }
}
```

And this is how it works. If we want to know the shortest path to a vertex, we have to follow back the 'previous' vertices from that vertex. Remember that this structure is built for a certain source vertex passed as parameter to the function. Values will be different if source vertex was another one.

Anyway, after visiting all vertices, we would have these values in our structure:

```
{
    <Vertex: A>: {'shortest': 7, 'previous': <Vertex: B>},
    <Vertex: B>: {'shortest': 5, 'previous': <Vertex: D>},
    <Vertex: C>: {'shortest': 7, 'previous': <Vertex: E>},
    <Vertex: D>: {'shortest': 0, 'previous': None},
    <Vertex: E>: {'shortest': 4, 'previous': <Vertex: F>},
    <Vertex: F>: {'shortest': 2, 'previous': <Vertex: D>}
}
```

With this we can answer the question what the minimum distance from vertex D to some of the vertices is. And it is also possible to build back the path from this vertex to the source D.

And for the last time (before the final exam), let's try to apply what we have seen.

Exercise!

Given the information you have received about Dijkstra's shortest path algorithm, implement a function that finds the shortest path from a source vertex to a destination vertex.

Don't continue until you have finished the exercise!

And here is a possible implementation of this algorithm:

```
def dijkstra_shortest_path(source_vertex, destination_vertex, graph):
    # Get all vertices
    unvisited_vertices = graph.get_vertices()
    # Create a dict table with the discovered shortest distances and paths
    shortest_path_table = { vertex: {'shortest': float('inf'), 'previous': None } for vertex
in unvisited_vertices}
    # Initialize the source vertex path distance as 0.
    shortest_path_table[source_vertex]['shortest'] = 0

    # Loop while there are untested vertices
    while unvisited_vertices:
        # Get the vertex with the minimum path distance until now
        current_vertex = min(unvisited_vertices, key=lambda vertex:
shortest_path_table[vertex]['shortest'])
        # Get its distance
        distance_from_source = shortest_path_table[current_vertex]['shortest']

        # Iterate over the adjacent vertices of this vertex that are still unvisited
        unvisited_adjacent_vertices = (v for v in
graph.get_adjacent_vertices(current_vertex) if v in unvisited_vertices)
        for adjacent_vertex in unvisited_adjacent_vertices:
            # Get incident edge between the vertices (current and adjacent)
            edge = graph.get_edge(current_vertex, adjacent_vertex)
            # Calculate what would be the distance to adjacent vertex through this current
vertex: Distance to current vertex + distance between these two vertices (edge's value)
            tentative_distance = distance_from_source + edge.value()
            # If the tentative distance is shorter than the already calculated, update the
table
            if tentative_distance < shortest_path_table[adjacent_vertex]['shortest']:
                shortest_path_table[adjacent_vertex]['shortest'] = tentative_distance
                shortest_path_table[adjacent_vertex]['previous'] = current_vertex

        # Remove the current vertex from the list of available vertices (as it is now
visited)
        unvisited_vertices.remove(current_vertex)
```

(Continue...)

```
    # Retrieve the path (in reverse order)
    path = []
    current = destination_vertex
    while current:
        path.append(current)
        current = shortest_path_table[current]['previous']

    # Return shortest distance and minimum distance path from source vertex to destination
vertex
    return (shortest_path_table[destination_vertex]['shortest'], path[::-1])
```

The time complexity of Dijkstra's shortest path algorithm depends on how it is implemented. More precisely which structure is used for the unvisited_vertices list. If a priority heap is used, then the time complexity is $O((n+m)\log n)$, being n the number of vertices and m the number of edges. If, like in our example, a Python sequence is used, then the time complexity is $O(n^2)$.

Note

The time complexity analysis is a little bit more challenging with this algorithm, so we will not enter into details this time.

The example algorithm presented in this chapter is focused on being as simple and understandable as possible. Performance is a secondary objective in this case.

Some useful Python resources

We have seen many different structures and programming strategies and techniques in this course. Some of them are used in Python. From Python's wiki page on time complexity, we can see how some of the Python's structures are implemented and their time complexities for different actions. Python lists are implemented with dynamic arrays, dicts and set are implemented with hash tables. It is not mentioned in that webpage, but strings are implemented with static arrays.

Python offers some modules by default that can help to use or build other structures. You don't need to install any external library, but you need to import the module. One common one is collections module, that "implements specialized container data types providing alternatives to Python's general purpose built-in containers". For example, it implements a doubly linked list structure as the `deque` object. Interesting module to investigate.

To work with more pure arrays, Python offers the array module, that offers arrays of different fixed-size types. For a implementation of a priority queue, there is the heapq module. For functionality to operate graph -like structures Python offers the graphlib module. And for a low level access to some Python structures and functionalities, there is the ctypes module, that provides C compatible data types, and allows calling functions in DLLs or shared libraries.

There are some popular external libraries (need to be installed separately) for Python, which are useful for scientific computing with Python and for working with large quantities of data. For example Numpy or Pandas. It is probably worth checking them out.